

INFORME DE INCIDENTE • PIPELINE INTRALE

Diez horas en vacío

Por qué el 8 de septiembre el pipeline consumió cuota todo el día, relanzó agentes sin sentido y produjo un solo PR.

2026-09-08 Ventana analizada: 00:00Z → 01:00Z (+1)

Fuentes: spawn-exit • cross-provider-dispatch • pulpo.log • liveness

No hubo una falla del modelo ni de los agentes. Fueron dos defectos de infraestructura que se combinaron: el barrido de huérfanos rebota trabajo sano después de cada reinicio del Pulpo, y el dispatch reintenta cada 30 segundos aunque no haya ningún provider capaz de atender. Uno produjo el retrabajo; el otro, el giro en vacío. Los dos están corregidos, con la verificación descrita al final.

EL DÍA EN NÚMEROS

Mucho movimiento, casi nada de avance

7538 eventos de dispatch	51 llegaron a lanzar un agente (0,7%)	1 PR mergeado a <code>main</code>	912 salidas de agente auditadas (2,2× el día anterior)
928 mensajes a Telegram	58% cuota semanal Anthropic <i>sin usar</i> al cierre		

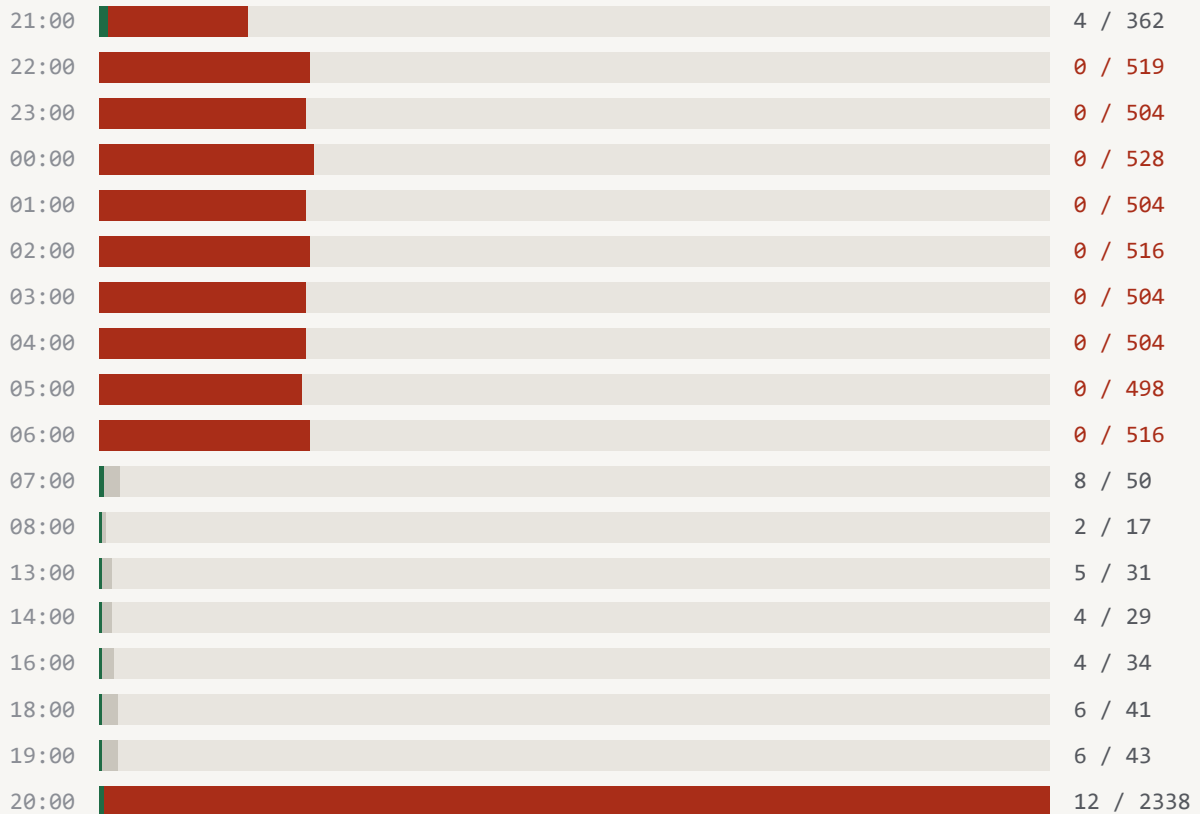
El contador de ola marca 93% de avance y el watchdog de dispatch atribuye el estancamiento a `causeKind: "quota"`. Ninguna de las dos cosas describe lo que

pasó: la cuota estaba disponible y el avance real fueron 9 PR el sábado, 1 el domingo y 1 el lunes.

RECONSTRUCCIÓN HORARIA

Dónde se fue el día

Cada barra es una hora. El largo es la cantidad de intentos de dispatch; el segmento verde, los que efectivamente lanzaron un agente. Hora de Buenos Aires.



■ giro en vacío (cadena de providers agotada) ■ operación normal ■ agentes efectivamente lanzados

Las horas 09:00–12:00, 15:00, 17:00 y 21:00 no registraron eventos de dispatch. La barra de las 20:00 es de esta misma noche: el ciclo **ya arrancó de nuevo** — 2338 intentos en una hora, 12 lanzados.

MECANISMO

Cómo un apagón nocturno se convirtió en trabajo rechazado

1 Anthropic entra en su ventana de reposo a las 20:00

```
Es
política
declarada,
no
un
bug:
provider-
schedule.json
apaga
Anthropic
de
20:00
a
07:00.
Verificado
en
vivo:
isProviderActiveNow('anthropic')
===
false .
```

2 La cadena de respaldo no tiene a nadie de pie

OpenAI/Codex
bloqueado
por
flag
de
cuota
(2190
veces),
Gemini
sin
licencia
de
CLI
(453),
Kimi
sin
credenciales
(86),

Cerebras
 agotado,
 NVIDIA
 NIM
 caído.
 La
 cadena
 anthropic
 →
 openai-
 codex
 se
 agotó
 1719
 veces.

3 El Pulpo reintenta cada 21 segundos, toda la noche

Al
 agotarse
 la
 cadena
 no
 hay
 backoff
 ni
 espera
 hasta
 la
 próxima
 ventana
 activa:
 vuelve
 a
 intentar
 de
 inmediato.
 ~170
 chain_exhausted
 por
 hora,

diez
horas
seguidas.

4 El dropfile entra a `trabajando/` ya vencido

`moveFile`
es
`fs.renameSync` ,
que
preserva
el
mtime.
Y el
barrido
mide
la
edad
de la
corrida
con
ese
mtime.
Un
archivo
que
esperó
horas
en
`pendiente/`
nace
por
encima
del
timeout,
aunque
su
agente
tenga
segundos

de
vida.

5 El reinicio del Pulpo borra el registro de quién está corriendo

`activeProcesses`

era
un
Map
en
memoria
sin
rehidratación.

Tras
un
boot
está
vacío:
ninguna
corrida
figura
como
viva,
ni
siquiera
las
que
están
corriendo
bien.

6 Las dos señales fallan juntas y el barrido rebota trabajo sano

Edad
vencida
más
registro
vacío
es
exactamente

la
 firma
 de
 "el
 agente
 murió".
 El
 Pulpo
 reinició
 a las
10:09:23
 y a
 las
10:09:43
 empezó
 a
 rebotar
 fases.
 Tres
 pasadas
 y
 quedan
 rechazadas.

7 De día se rehace desde cero

De
 ahí
 los
 attempt-
 2 en
 guru,
 po,
 ux y
 pipeline-
 dev
 sobre
 los
 mismos
 7

issues,
y los
~113
M
de
tokens
(40%
del
día)
gastados
en
repetir
trabajo
ya
hecho.

```
10:09:23 [pulpo] reinicio
10:09:43 [huerfanos] 5801.pipeline-dev lleva 592min sin proceso → pendiente/ (intento 1/3)
10:35:00 [huerfanos] 5801.pipeline-dev lleva 617min sin proceso → pendiente/ (intento 2/3)
10:56:29 [huerfanos] 5801.pipeline-dev lleva 638min sin proceso → pendiente/ (intento 3/3)
11:09:55 [huerfanos] 5801.pipeline-dev excedió 3 reintentos → rechazado
```

Los tres reintentos se consumieron en una hora de la mañana, no durante la noche – y la edad crece 592 → 617 → 638 porque el mtime nunca se refresca. Le pasó lo mismo a #6239, con `qa:passed` puesto.

FALLAS DE FONDO

Las seis cosas que hay que arreglar

CORREGIDO

El barrido de huérfanos rebotaba trabajo sano tras cada reinicio

Las dos señales con las que decide fallan juntas después de un boot. Ahora el registro de corridas se persiste y se rehidrata revalidando cada PID contra el sistema operativo; la entrada a `trabajando/` marca el mtime, así la edad mide la

corrida y no el dropfile; y mientras el registro esté frío el barrido se abstiene, porque no puede distinguir "murió" de "no lo lancé yo".

registro frío tras el reinicio: el barrido NO rebota la corrida	✓
pasada la ventana de gracia, una corrida sin proceso sí se recupera	✓
la entrada a trabajando/ refresca el mtime	✓

`.pipeline/lib/orphan-guard.js` · `.pipeline/lib/active-process-registry.js`

CORREGIDO

El dispatch no esperaba cuando no había a quién despachar

Con toda la cadena apagada, el Pulpo reintentaba cada 30 segundos. Ahora cada agotamiento programa una espera exponencial de 1 a 15 minutos para ese (skill, issue), consultada *antes* de mover el dropfile — así tampoco rebota entre carpetas en cada vuelta. Medido sobre la noche real: **1200 intentos pasan a 44, un 96,3% menos.**

```
chain_exhausted: 2143   reason: "all_gated"
fallback_also_gated: 2672   primary_inactive_by_schedule: 2133
fallback_selected: 51
```

`.pipeline/lib/dispatch-backoff.js`

PENDIENTE

Un provider bloqueado por cuota no puede desbloquearse solo

El flag de cuota se limpia únicamente cuando ese provider tiene un spawn exitoso — pero mientras el flag está puesto, el provider no puede spawnear. Es un candado circular: OpenAI/Codex bloqueó 2190 dispatches con el flag puesto, y hoy su cuota real marca 0% de uso. Cerebras está marcado agotado hasta mañana a las 23:00Z por una detección de las 20:00.

```
quota-exhausted.json → cerebras: exhausted:true, resets_at 2026-09-09T23:00:29Z
multi-provider-health → openai: quota pct 10, state green
```

`.pipeline/lib/quota-exhausted.js` — `clearFlag` sólo en `success_spawn`

ALTA

La telemetría de diagnóstico está envenenada

El clasificador marca como error transitorio *toda* corrida que dure más de 30 segundos y no traiga un error reconocible — es decir, todos los agentes sanos,

que corren entre 5 y 27 minutos. Resultado: 809 de 912 salidas auditadas como `transient_5xx`, 682 de ellas con `exit_code: 0` y muerte normal. Verificado que **no dispara relanzamientos**: el Pulpo decide el éxito por `exit_code`. Pero convierte en basura la única fuente que sirve para diagnosticar incidentes de provider — que es parte de por qué no aparecía una causa clara.

```
if (timedOut === true || durationMs >= 30000) return { errorClass: 'transient_5xx' }
```

`.pipeline/lib/agent-launcher/provider-error-parser.js:814 - classifyByContext`

ALTA

El diagnóstico automático apunta al lugar equivocado

El watchdog de dispatch reporta `causeKind: "quota"` con Anthropic al 58% semanal y 76% de sesión disponibles. La causa real es de agenda y disponibilidad de cadena, no de cuota. Mientras el sistema se autodiagnostique mal, cada incidente arranca la investigación por la pista falsa.

MEDIA

El ruido de Telegram tapa la señal

928 mensajes en el día, de los cuales 177 son alertas del propio servicio de Telegram sobre sí mismo. Se emitieron además 2112 eventos de pausa por agotamiento — uno por cada vuelta del bucle — y varios fallaron al enviarse. Hay que agrupar por episodio, no por evento.

DESCARTADO CON EVIDENCIA

Lo que no fue

- **No fue cuota agotada.** Anthropic cerró el día con 58% semanal y 76% de sesión disponibles.
- **No fue calidad del código ni rebotes de QA.** Un solo rebote en todo el día, y de infraestructura. #5801 y #6239 tienen `qa:passed`.
- **No fue el watchdog matando al Pulpo.** Hubo un solo `kill-respawn` en todo el día, a las 21:45 — consecuencia del final del día, no su causa.
- **No fue el modelo ni los agentes.** Cuando lograron correr, terminaron con salida normal; el problema fue que su trabajo se descartaba después.

PLAN DE SALIDA

Qué hacer, en este orden

Hecho · PR sobre #7103

con tests

1. El registro de corridas sobrevive al reinicio.

Se persiste en cada cambio y se rehidrata al arrancar, revalidando cada PID contra el sistema operativo: lo que ya murió no vuelve. Extiende `Map`, así que ningún call-site existente cambió.

2. La entrada a `trabajando/` marca el `mtime`.

La edad pasa a medir la corrida y no la antigüedad heredada del dropfile. Va dentro de `moveFile`, el único lugar por el que pasan todos los movimientos, para que ningún call-site futuro tenga que acordarse.

3. El barrido se abstiene mientras no pueda saber.

Si el registro no es confiable —primer arranque tras el deploy, o archivo corrupto— su vacío no prueba que nadie esté corriendo, y el barrido espera en vez de rebotar. Con un registro rehidratado la espera no aplica: lo que no figura, no vive.

4. Backoff exponencial ante cadena agotada.

De 1 a 15 minutos por `(skill, issue)`, consultado antes de mover el dropfile. Se reinicia en cuanto hay un despacho efectivo.

Queda pendiente

issues aparte

1. Decidir qué pasa de noche, explícitamente.

Hay una contradicción de política: se apaga Anthropic 11 horas y se deja al pipeline intentando igual, con una cadena de respaldo que en la práctica no existe. Las dos salidas honestas son conseguir un respaldo nocturno que funcione de verdad, o declarar la noche como parada. El backoff contiene el daño, no resuelve la contradicción.

2. Romper el candado circular del flag de cuota.

Que el flag caduque por `resets_at` revalidado contra el adaptador, sin depender de un spawn exitoso que el propio flag impide.

3. Arreglar `classifyByContext` .

Que “duró más de 30 segundos” deje de significar error. Exigir `timedOut` o salida distinta de cero. Sin esto, el próximo incidente vuelve a investigarse a ciegas.

4. Corregir la atribución de causa y agrupar las notificaciones.

Si la cuota está disponible, la causa no es cuota. Y un aviso por episodio de agotamiento, no uno por vuelta del bucle.

VERIFICACIÓN

Cómo sabemos que salimos

La comprobación tiene tres niveles, del más rápido al más definitivo. Los dos primeros ya están corridos.

1 · Los tests fijan cada pieza

25 tests nuevos, incluidos tres que ejercitan el **barrido real** del Pulpo —no una réplica— contra el escenario exacto del incidente: dropfile con 592 minutos de mtime heredado y registro frío.

2 · La noche del incidente, replicada

Un test simula las diez horas reales con la cadena caída y el Pulpo tickeando cada 30 segundos, y cuenta los intentos que habría hecho con el backoff puesto:

```
ticks de la noche (sin backoff): 1200
intentos con backoff           :    44
reducción                      : 96,3%
```

3 · La medición en producción

Los tests prueban las unidades; esto mide el pipeline real, sobre los mismos archivos con los que se diagnosticó el incidente. Se corre mañana:

```
node .pipeline/scripts/verificar-fix-huerfanos.js
```

SEÑAL	8 DE SEPTIEMBRE	OBJETIVO
Intentos de dispatch por día	7538	< 500
Tasa de dispatch efectivo	0,7%	> 60%

SEÑAL	8 DE SEPTIEMBRE	OBJETIVO
Fases rechazadas por huérfano	3	0
Corridas de retrabajo (<code>attempt ≥ 2</code>)	26	< 3

El script trae los umbrales fijados de antemano y la línea base del 8 de septiembre, y sale con código distinto de cero si alguno queda en rojo — la verificación no depende de leer un log a ojo. Está validado: corrido contra el día del incidente reproduce exactamente esos cuatro números.

Elaborado el 2026-09-08 sobre `spawn-exit-2026-09-0*.jsonl`, `cross-provider-dispatch-2026-09-08.jsonl`, `pulpo.log`, `pulpo-liveness.log`, `service-liveness.log`, `svc-telegram.log` y el estado del kanban en `.pipeline/desarrollo/`.